

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Example 3.4 Precision and recall accuracy for the MNIST dataset
5  Machine Learning for Engineering Problem Solving
6  @author: Austin Downey
7  """
8
9  import IPython as IP
10 IP.get_ipython().run_line_magic('reset', '-sf')
11
12 import numpy as np
13 import scipy as sp
14 import matplotlib.pyplot as plt
15 import sklearn as sk
16 from sklearn import linear_model
17 from sklearn import datasets
18 from sklearn import metrics
19
20 cc = plt.rcParams['axes.prop_cycle'].by_key()['color']
21 plt.close('all')
22
23
24 %% Load your data
25
26 # Fetch the MNIST dataset from openml
27 mnist = sk.datasets.fetch_openml('mnist_784', as_frame=False, parser='auto')
28 X = np.asarray(mnist['data']) # load the data
29 Y = np.asarray(mnist['target'], dtype=int) # load the target
30
31 # Split the data set up into a training and testing data set
32 X_train = X[0:60000,:]
33 X_test = X[60000:,:]
34 Y_train = Y[0:60000]
35 Y_test = Y[60000:]
36
37 %% Train a Stochastic Gradient Descent classifier
38
39 # Extract a subset for our "5-detector".
40 Y_train_5 = (Y_train == 5)
41 Y_test_5 = (Y_test == 5)
42
43 # build and train the classifier
44 sgd_clf = sk.linear_model.SGDClassifier()
45 sgd_clf.fit(X_train, Y_train_5)
46
47 %% Build the Confusion Matrices
48
49 # Return the predictions made with the trained model
50 X_train_pred = sgd_clf.predict(X_train)
51
52 # build the confusion Matrix
53 confusion_matrix = sk.metrics.confusion_matrix(Y_train_5, X_train_pred)
54
55 TN = confusion_matrix[0,0]
56 FP = confusion_matrix[0,1]
57 FN = confusion_matrix[1,0]
58 TP = confusion_matrix[1,1]
59
60 %% Calculate Precision and Recall
61
62 # calculate the Precision and Recall values using the commands discussed in class
63 accuracy = (TP + TN)/(TP + TN + FP + FN)
64 precision = TP/(TP+FP)
65 recall = TP/(TP+FN)
66
67 # of course, SK learn has built-in functions for this.

```

```

68 accuracy_SK = sk.metrics.accuracy_score(Y_train_5, X_train_pred)
69 precision_SK = sk.metrics.precision_score(Y_train_5, X_train_pred)
70 recall_SK = sk.metrics.recall_score(Y_train_5, X_train_pred)
71
72 # compute the F1 score for the data set
73 F1 = sk.metrics.f1_score(Y_train_5, X_train_pred)
74
75 %% plot the precision and recall over the threshold domain.
76
77 # first, compute the scores for the predictions.
78 Y_scores= sgd_clf.decision_function(X_train)
79
80 #Y_scores= Y3
81 # now, for the scores and the target training set, calculate the precisions, recalls,
threshold values.
82 precisions, recalls, thresholds = sk.metrics.precision_recall_curve(Y_train_5, Y_scores)
83
84 # now, compute the F1 score over the entire threshold range
85 F1s = 2/(1/precisions + 1/recalls)
86
87 # plot the Precision and Recall vs threshold.
88 plt.figure()
89 plt.plot(thresholds, precisions[:-1], "--", label="Precision")
90 plt.plot(thresholds, recalls[:-1], "-", label="Recall")
91 plt.plot(thresholds, F1s[:-1], ":", label="F1 score")
92 plt.xlabel("Threshold")
93 plt.ylabel("normalized precision\nand recall index")
94 plt.legend(loc=6, framealpha=1)
95 plt.ylim([-0.05, 1.05])
96 plt.grid(True)
97 plt.tight_layout()
98
99
100
101
102
103
104
105
106
107
108
109

```